

Brody Validation Report

Full validation of the Brody repository: syntax, correctness, performance, styling and accessibility, delivery formats and deployment.

Run on 2026-09-20 against the working tree, macOS, Node 25, Chrome, Docker Desktop.

Results come from commands that were actually run; nothing here is estimated.

Contents

1. Verdict	2
2. Method	2
3. Syntax and static validation	2
4. Functional validation	2
5. Self-analysis: Brody analysing Brody	3
6. Performance	4
7. Styling and accessibility	5
8. Viewing and downloading results	6
9. Deployment	6
10. Not verified and known limitations	7

1. Verdict

Area	Result	Evidence
Syntax and types	Pass	tsc --noEmit clean, ESLint 0 problems, production build passes with Turbopack and webpack
Functional correctness	Pass	125 unit and integration tests in 12 files, all passing
Browser end to end	Pass	10 Playwright tests in Chrome, all passing
Performance	Pass after fixes	3 bottlenecks found and removed; budgets enforced by tests
Styling and layout	Pass after fixes	4 viewports x light and dark, no overflow, no console errors
Accessibility	Pass after fixes	axe WCAG 2 A and AA, 0 serious or critical violations on every view
Downloads (PDF, Word, Markdown)	Pass	Generated and parsed back with independent PDF and DOCX readers
Docker	Pass	Image builds, healthy, non-root, exports work inside the container
Live AI provider	Not verified	The supplied key was rejected: "not scoped to a workspace"

2. Method

1. Static checks: TypeScript compiler, ESLint, both Next.js bundlers.
2. Automated tests: Vitest for logic, API handlers, security, exports and the analysis pipeline. Playwright for the real UI.
3. Self-analysis: Brody analysed its own repository with its own pipeline and analyzers. Findings that turned out to be analyzer mistakes were fixed in the analyzers, and genuine findings were fixed or recorded.
4. Performance: a synthetic-repository benchmark (200, 1,000 and 3,000 source files), CPU profiling of the slowest operation, and web timing budgets in Playwright.
5. Styling: screenshots reviewed by eye at four widths in both colour schemes, plus automated overflow, console-error and axe checks.
6. Deployment: Docker Compose build and run, with a real upload and export inside the container.

3. Syntax and static validation

Check	Result
npm run typecheck	No errors, including tests, e2e specs and scripts
npm run lint	0 errors, 0 warnings
next build (Turbopack)	Compiles, 31 routes generated
next build --webpack (used by Docker)	Compiles with no build warnings after removing a dynamic createRequire
Brody's TypeScript syntax analyzer on Brody	0 syntax errors in 117 source files
Brody's ESLint adapter on Brody	0 findings
Python ast and Ruff on the repository's Python	0 findings (the only Python is the test fixture)

4. Functional validation

125 tests across 12 files. All pass in about 15 seconds.

Suite	Tests	What it proves
Ingestion and security	28	Single file, files, folder, ZIP, GitHub URL forms, ignore rules, binaries, malformed archives, zip-slip, symlinks, decompression bombs, size and entry limits, token handling

Suite	Tests	What it proves
Parsing	6	Symbols, imports, calls, inheritance and routes for TypeScript, Python, Go, Java, C#, Ruby, Rust, PHP, SQL, Prisma, Markdown, YAML, text fallback
Secrets	4	Detection, placeholders, masking, encryption, log scrubbing
Review	14	Structured finding validation, evidence re-anchoring, fabricated evidence rejected, patch validation, dedupe, codes, rule engine
Pipeline	10	All 12 stages on the fixture, architecture, routes, models, flows, incremental analysis, cancellation, recovery
AI with scripted provider	10	Grounded and invented findings, documentation hierarchy, conflicts, injection stays inside untrusted blocks, secrets never reach the model, cache reuse, provider failure
Provider clients	8	Structured output request shape, fallback degradation, refusals, truncation retry, error explanations
API	11	Every route, valid and invalid input, 404, 409, uploads, deletion
Map, retrieval, Q&A, report safety	10	Graphs, change impact, hybrid search, exact answers with citations, HTML escaping
Analyzers and semantic search	5	TypeScript, ESLint, Python and Ruff adapters never execute repository code
Precision	11	Analyzer false positives found by self-analysis stay fixed
Exports	8	PDF and Word validity, scopes, page numbers, glyphs, caching

Browser tests in Chrome (10 tests) cover the whole workflow from ZIP upload to every view, the downloads, four viewports in two colour schemes, accessibility, keyboard use, and performance budgets.

5. Self-analysis: Brody analysing Brody

Measured run: 174 files, 29,057 lines, 908 symbols, 3,018 relationships, 34 routes, 12 data models, in 9.3 seconds at 594 MB peak memory. The generated reports were added to the repository afterwards, so a later run sees a few more files. The final run reports 21 findings.

The first run reported 82 findings, most of them wrong. Each class of false positive was fixed at the source:

Problem in the analyzer	Effect on Brody	Fix
Rules matched text inside strings, regex literals and comments	17 false "eval" findings	Rules now match code only, with template interpolations preserved
RegExp .exec() treated as shell exec	6 false high findings	Only real shell exec forms match
Any function named guard counted as authentication	Reported 23 of 25 routes as authenticated	Authentication needs real auth patterns
Fixture and test manifests counted as project dependencies	53 external services reported, mostly from a data table	Test data is ignored; URL evidence needs a real URL and is skipped in catalog-like files
Everything under lib/ became one area	38 files in "Shared Utilities"	Areas are named after the module under lib/
setInterval in a component reported as a scheduled job	3 false entry points	Only cron libraries or worker modules count
Secrets in test files reported Critical	5 false Critical findings	Test and fixture secrets are Low

Result: 82 findings became 21 (0 Critical, 0 High, 8 Medium, 13 Low), and the detected services went from 53 to the 4 Brody really uses. The architecture is now labelled "full-stack monolith (Next.js)".

Genuine issues found in Brody and fixed:

Issue	Fix
Reading a project folder walked all of node_modules (38,531 files) before exclusions, hitting the file limit	Dependency and build folders are counted, not read; the browser folder picker skips them too, with an opt-in checkbox

Issue	Fix
/api/status used blocking spawnSync on every call	Asynchronous probes, cached for a minute
Analyzer temp files were written with blocking calls inside the server process	Asynchronous writes that yield to the event loop
Mermaid SVG assigned through innerHTML	Parsed, scripts and event handlers removed, inserted as DOM nodes
.env.example was git-ignored, so it would never be committed	Added the negation to .gitignore
No CI	Added a GitHub Actions workflow: typecheck, lint, tests, build, browser tests
Docker build ran out of memory on a 1.9 GB VM	Webpack builder, capped heap and workers

Remaining findings, all accepted:

Finding	Assessment
8 Medium: high cyclomatic complexity in detectRoutes (190), buildGraph (151), extractTsJs (166), detectEntryPoints, enhanceWithAI, extractPython, FilesPage, Home	Real technical debt. These are long tables of framework rules and large UI components. The tests around them are strong, so splitting is a safe future task.
6 Low: long functions and one long component in the same places	Same cause
1 Low: none of the 25 API routes has an authentication check	True and documented. Brody is single-tenant and should sit behind an authenticating proxy.
4 Low: secret-like values in tests	Intentional fake credentials used to test detection
1 Low: fixture credential	Intentional, documented in the fixture README
1 Low: 3 important files without direct tests	The React client helpers, covered indirectly by the browser tests

6. Performance

6.1 Analysis pipeline

Synthetic TypeScript repositories, no AI. Times in milliseconds.

Source files	Lines	Parse	Graph	Index	Architecture	Static analysis	Whole pipeline	Peak memory
200	8,169	369	244	79	84	366	1,263	491 MB
1,000	41,228	1,296	1,011	271	320	590	3,737	620 MB
3,000	125,316	3,871	3,212	940	1,157	1,614	11,565	971 MB

Scaling is close to linear: 15 times the files took 9 times as long. A second analysis of an unchanged repository reuses cached parses and AI explanations.

6.2 Interactive operations at 3,000 files

Operation	Time
Change impact for a symbol	18 ms
Search, cold index	504 ms
Search, warm	56 ms
Markdown report	241 ms
PDF, full report	1.5 s
Word, full report	0.6 s

6.3 Bottlenecks found and removed

Bottleneck	Before	After	Cause
PDF generation	22.5 s	2.0 s	pdfkit re-read and re-parsed the same font file 12,559 times because it was also passed as the default font
Search on a large index	697 ms	56 ms	A full scan built term maps per query; now an inverted index and cached graph adjacency
Graph build	Quadratic lookup per file	Linear	A list search was replaced by a map
Architecture page JavaScript	816 KB	237 KB	Mermaid loaded eagerly; the area graph now reuses the Code Map canvas and Mermaid loads only when the ER diagram scrolls into view

6.4 Web performance, production build

Route	DOMContentLoaded	Largest paint	JavaScript transferred
Home	60 ms	100 ms	145 KB
Overview	95 ms	304 ms	160 KB
Code Review	38 ms	276 ms	159 KB
System Explanation	32 ms	276 ms	160 KB
Architecture	73 ms	288 ms	237 KB
Code Map	41 ms	260 ms	236 KB
Files	67 ms	304 ms	187 KB
Ask	51 ms	180 ms	159 KB
Reports	50 ms	204 ms	157 KB

Budgets enforced by the tests: DOMContentLoaded under 2.5 s, largest paint under 3.5 s, JavaScript under 700 KB. All API endpoints answered in 4 to 9 ms on a warm server against an 800 ms budget.

7. Styling and accessibility

Pages checked: Home, Overview, Code Review, System Explanation, Architecture, Code Map, Files, Ask, Reports. Viewports: 1440, 1100, 820 and 390 pixels wide. Both light and dark colour schemes.

Check	Result
Horizontal page overflow	None at any width
Browser console errors	None
axe-core, WCAG 2 A and AA, 2.1 AA	0 serious or critical violations on every view, light and dark
Keyboard	Search shortcut, menus open and close with Escape, splitters focusable with arrow keys
Screen-reader structure	Landmarks, labelled controls, visible focus, status and alert roles

Defects found by this testing and fixed:

Defect	Fix
Phone layout unusable: the side navigation took half the screen and panes ran off-screen	Navigation becomes a scrolling tab bar; Review, Files and Code Map stack vertically below 1024 px
Tablet Review pane squeezed to one word per line	Same responsive stacking
Menu roles used outside a menu (critical)	Roles only inside real menus
Links inside expandable headers (nested interactive)	Links moved into the expanded body

Defect	Fix
Splitters missing required value attributes (critical)	Added current, minimum and maximum values
Severity colours and code-comment colour below contrast in light mode	Darkened the palette and the syntax theme
Primary buttons and diagram text below contrast in dark mode	Dedicated button foreground token and explicit diagram theme colours
Scrollable code and diagram regions not keyboard-focusable	Made focusable
Links in running text distinguished only by colour	Underlined
A custom stylesheet overrode utility classes (inputs full width, nav underlined on hover)	Moved custom styles into cascade layers

8. Viewing and downloading results

Every result view now offers the result as PDF, Word or Markdown, and there is a Reports page listing them all.

Where	What is offered
Header, every page	Export report: the full report
Overview	A "Your report is ready" strip with PDF, Word and Markdown
Code Review	Review report
System Explanation	Explanation
Architecture	Architecture and code map
Code Map	Detailed code map
Ask Repository	Question and answer transcript, plus Copy as Markdown on each answer
Reports page	Six reports, each as PDF, Word, Markdown and web page; the full report also as JSON
Project list	A Report menu for every finished project
Command line	<code>npm run analyze</code> writes PDF, Word, Markdown, HTML and JSON; <code>npm run convert</code> converts any Markdown file

Each format has View and Download where the browser can show it. Downloads show progress and report errors in place. PDF opens in a new tab, Markdown opens as plain text, Word downloads.

Verification of the files themselves:

Format	How it was checked
PDF	Parsed with pdf.js: page count, section text, legend symbols, bookmarks for all 19 sections, contents page numbers match the pages sections start on. Pages were rendered to images and reviewed by eye.
Word	Opened with the Mammoth document reader: real heading styles, tables, bullet lists, monospaced code, no conversion errors
Markdown	Section order and numbering asserted; names it <code>CODEBASE_REPORT.md</code>

The PDF has a cover, a contents list with page numbers, bookmarks and running footers. Symbols that the main font lacks are drawn from fallback fonts, so every legend glyph renders.

9. Deployment

Check	Result
<code>docker compose up --build</code>	Builds on a 1.9 GB Docker VM, container healthy
User	Non-root, all capabilities dropped
Address	Answers at <code>brody:3003</code> ; <code>localhost:3003</code> refused with 421

Check	Result
Data	Survives a container restart
Exports inside the container	PDF (26 pages), Word and Markdown generated correctly

10. Not verified and known limitations

1. **Live AI calls.** The supplied Anthropic key was rejected on every request with "This API key is not scoped to a workspace". The failure is handled and shown clearly, but the Anthropic and OpenAI-compatible request paths are verified only against test doubles. Set `ANTHROPIC_WORKSPACE_ID` and run `npm run verify:ai`.
2. Private GitHub import is verified only against a mocked GitHub API. A public repository was imported live.
3. The interface was verified in desktop Chrome and at phone and tablet widths in Chrome. Safari and Firefox were not tested.
4. `mypy` and `go vet` are deliberately not run, because they can execute repository code outside a sandbox.
5. Memory grows with repository size: about 1 GB at 3,000 files. Very large repositories need a machine with more memory or a smaller upload.
6. Brody has no user authentication. Run it locally or behind an authenticating proxy.